

Finding Shortest Paths

Definition:

Let $G = (V, E)$ be a weighted digraph (=directed graph), with weight function $w: E \rightarrow \mathbb{R}$.

1. The **weight of a path** $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its edges, i.e.,

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

Finding Shortest Paths

Definition:

Let $G = (V, E)$ be a weighted digraph (=directed graph), with weight function $w: E \rightarrow \mathbb{R}$.

1. The **weight of a path** $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its edges, i.e.,

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

2. The **weight of a shortest path from u to v** is:

$$\delta(u, v) = \begin{cases} \min_{p \text{ from } u \text{ to } v} w(p), & \text{if path exists} \\ \infty, & \text{otherwise} \end{cases}$$

Finding Shortest Paths

Definition:

Let $G = (V, E)$ be a weighted digraph (=directed graph), with weight function $w: E \rightarrow \mathbb{R}$.

1. The **weight of a path** $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its edges, i.e.,

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

2. The **weight of a shortest path from u to v** is:

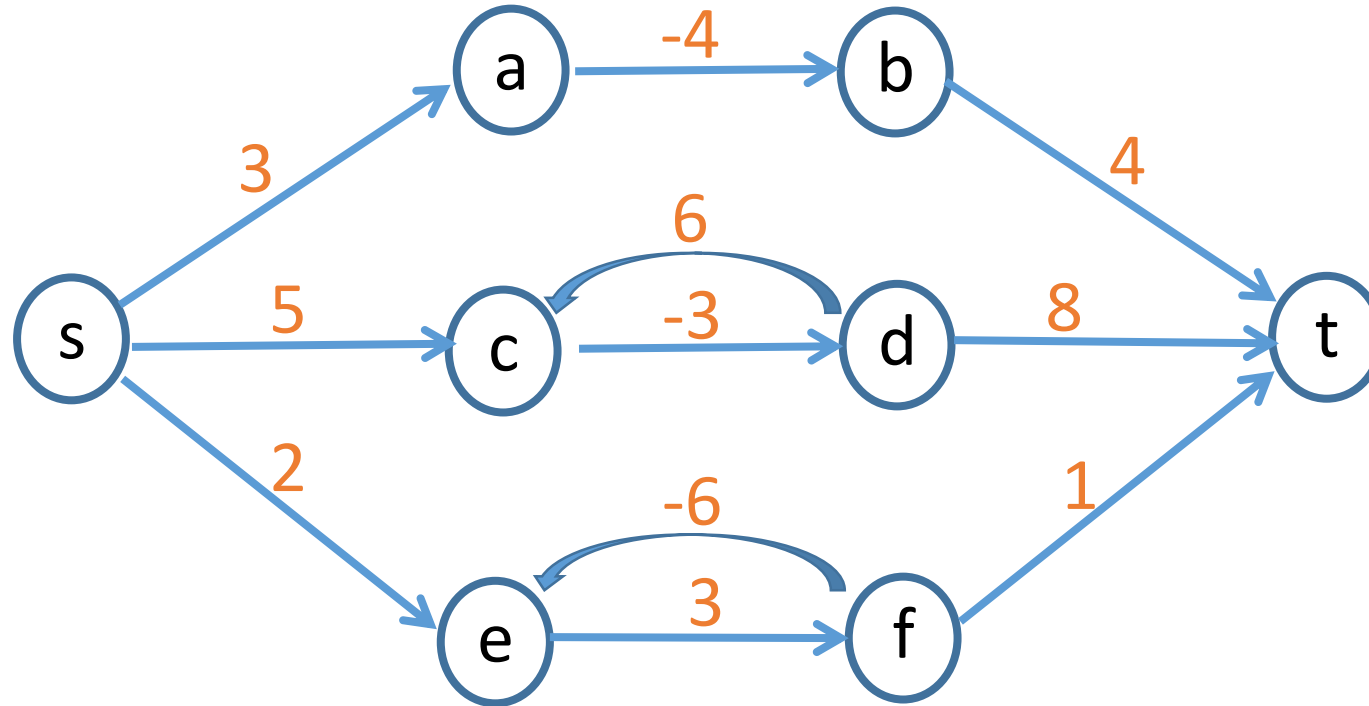
$$\delta(u, v) = \begin{cases} \min_{p \text{ from } u \text{ to } v} w(p), & \text{if path exists} \\ \infty, & \text{otherwise} \end{cases}$$

3. **A shortest path from u to v** is a path p from u to v such that:

$$w(p) = \delta(u, v).$$

Finding Shortest Paths

Example (negative weights):



If there is a path from s to v with negative weight cycle then
$$\delta(s, v) = -\infty.$$

Finding Shortest Paths

Theorem (optimality of shortest paths sub-paths):

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.

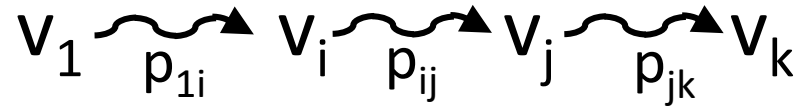
Let $p = \langle v_1, \dots, v_k \rangle$ be a shortest path from v_1 to v_k . Also, for every $1 \leq i \leq j \leq k$ let $p_{ij} = \langle v_i, \dots, v_j \rangle$ be the sub-path of p from vertex v_i to vertex v_j .

Then, p_{ij} is a shortest path from v_i to v_j .

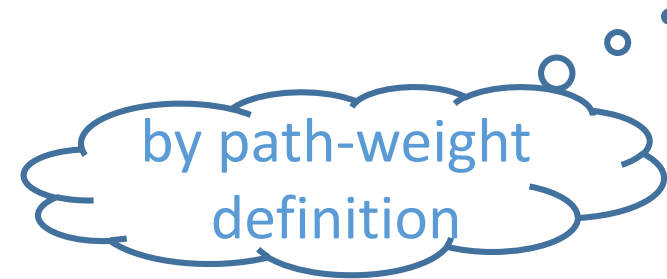
Finding Shortest Paths

Proof:

Let us split the path p to the following sub-paths:



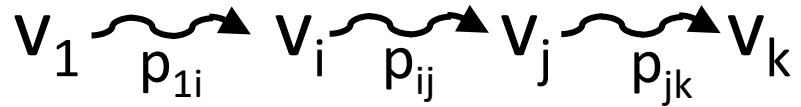
Then, $w(p) = w(p_{1i}) + w(p_{ij}) + w(p_{jk})$.



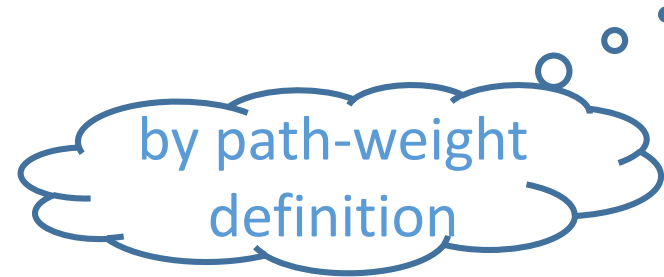
Finding Shortest Paths

Proof:

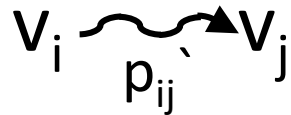
Let us split the path p to the following sub-paths:



Then, $w(p) = w(p_{1i}) + w(p_{ij}) + w(p_{jk})$.



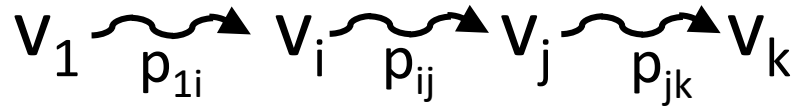
Assume to the contrary that there exists a path p_{ij}' from v_i to v_j such that: $w(p_{ij}') < w(p_{ij})$.



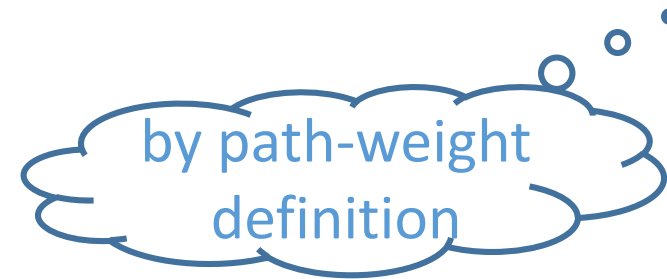
Finding Shortest Paths

Proof:

Let us split the path p to the following sub-paths:



Then, $w(p) = w(p_{1i}) + w(p_{ij}) + w(p_{jk})$.



Assume to the contrary that there exists a path p_{ij}' from v_i to v_j such that: $w(p_{ij}') < w(p_{ij})$. Then, we can construct a path p' from v_1 to v_k :

A diagram showing a sequence of vertices v_1, v_i, v_j, v_k connected by wavy lines representing paths. The first segment from v_1 to v_i is labeled p_{1i} , the second from v_i to v_j is labeled p_{ij}' , and the third from v_j to v_k is labeled p_{jk} . Each segment has a small black triangle pointing to the right, indicating the direction of the path.

with weight:

$w(p') = w(p_{1i}) + w(p_{ij}') + w(p_{jk}) < w(p)$, contradiction.

Finding Shortest Paths

Conclusion:

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.

Assume that a shortest path from s to v can be split to



for some vertex u and sub-path p' .

Then, the weight of a shortest path from s to v is:

$$\delta(s, v) = \delta(s, u) + w(u, v).$$

SSSP – Single Source Shortest Paths

Properties of RELAX:

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.

1. Let $(u, v) \in E$. Then, right after the call $\text{RELAX}(u, v, w)$ it holds:
$$d[v] \leq d[u] + w(u, v).$$

SSSP – Single Source Shortest Paths

Properties of RELAX:

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.

1. Let $(u, v) \in E$. Then, right after the call $\text{RELAX}(u, v, w)$ it holds:
$$d[v] \leq d[u] + w(u, v).$$
2. Let $s \in V$ be the source, and assume the graph was initialized by ISS call. Then $d[v] \geq \delta(s, v)$, and this invariant holds for any sequence of RELAX calls on the edges of G . Moreover, when $d[v]$ reaches its lower-bound $\delta(s, v)$, its value never changes.

SSSP – Single Source Shortest Paths

Properties of RELAX:

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.

3. Let $s \in V$ be the source, and let $s \rightsquigarrow u \rightarrow v$ be a shortest path for some u, v in G . Assume G was initialized by ISS call, and then a sequence of RELAX was performed including the call $\text{RELAX}(u, v, w)$. If $d[u] = \delta(s, u)$ some time before this call, then $d[v] = \delta(s, v)$ **any time** after this call.

SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Cost

Command Number	Linear Queue	Binary Heap
1	$\Theta(V)$	$\Theta(V)$
2	$\Theta(1)$	$\Theta(1)$
3	$\Theta(V)$	$\Theta(V)$
4x{5}	$\Theta(V ^2)$	$O(V \log V)$
4x{6}	$\Theta(V)$	$\Theta(V)$
4x{7-8}	$\Theta(E)$	$O(E \log V)$
Total	$\Theta(V ^2)$	$O((E + V) \log V)$

SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

Theorem:

If Dijkstra's algorithm runs on a weighted digraph $G = (V, E)$ with $w: E \rightarrow \mathbb{R}^+$ and a source s . Then, when the algorithm halts it holds that

$$d[u] = \delta(s, u)$$

For every $u \in V$.

SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

Proof:

We show that for every $u \in V$, when the algorithm inserts it to the set S , it holds that $d[u] = \delta(s, u)$, and this equality persists until algorithm halts.

SSSP- Single Source Shortest Paths

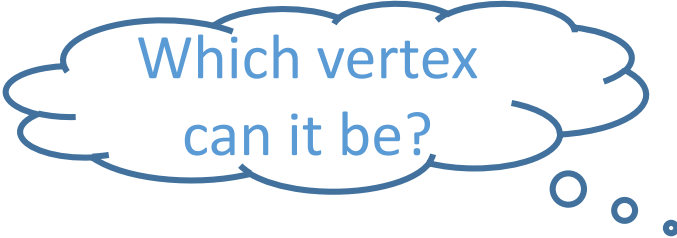
Dijkstra's Algorithm Correctness

Proof:

We show that for every $u \in V$, when the algorithm inserts it to the set S , it holds that $d[u] = \delta(s, u)$, and this equality persists until algorithm halts.

Assume to the contrary that this does not hold, and let u be the first vertex for which when it was inserted to S , we have:

$$d[u] \neq \delta(s, u).$$



Which vertex
can it be?

SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

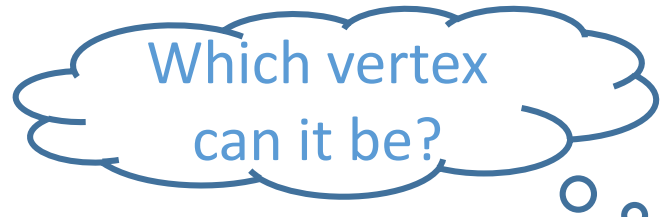
Proof:

We show that for every $u \in V$, when the algorithm inserts it to the set S , it holds that $d[u] = \delta(s, u)$, and this equality persists until algorithm halts.

Assume to the contrary that this does not hold, and let u be the first vertex for which when it was inserted to S , we have:

$$d[u] \neq \delta(s, u).$$

Necessarily, $u \neq s$,
since s is the first vertex that was inserted to S , and then we had:
 $d[s] = \delta(s, s) = 0$.



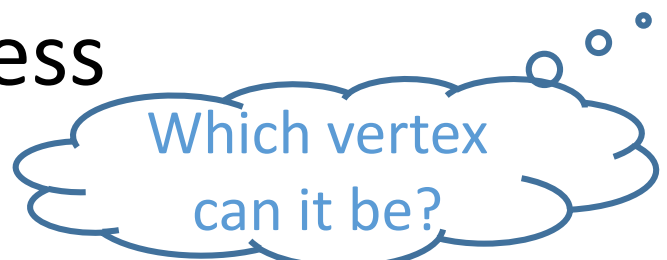
Which vertex
can it be?

SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

Proof (cont.):

Since $u \neq s$, we know that before u was inserted to S , we had:
 $S \neq \emptyset$.



Which vertex
can it be?

SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

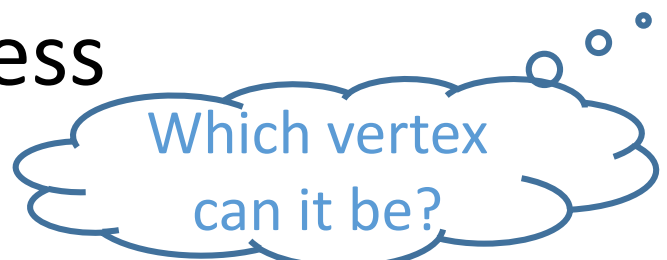
Proof (cont.):

Since $u \neq s$, we know that before u was inserted to S , we had:
 $S \neq \emptyset$.

Also, there must be a path from s to u , since otherwise we have:

$d[u] = \delta(s, u) = \infty$, contrary to the assumption that:

$$d[u] \neq \delta(s, u).$$

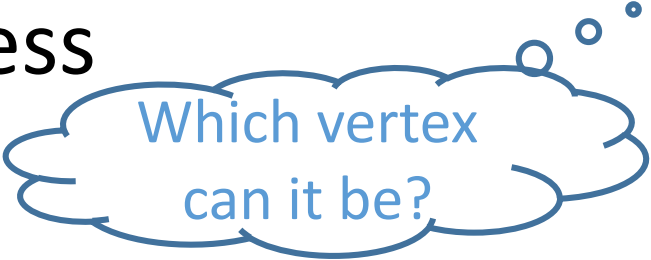


Which vertex
can it be?

SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

Proof (cont.):



Which vertex
can it be?

Since $u \neq s$, we know that before u was inserted to S , we had:
 $S \neq \emptyset$.

Also, there must be a path from s to u , since otherwise we have:
 $d[u] = \delta(s, u) = \infty$, contrary to the assumption that:
 $d[u] \neq \delta(s, u)$.

Since there exists a path from s to u , a shortest path is defined.
Let p be such shortest path from s to u .

SSSP- Single Source Shortest Paths

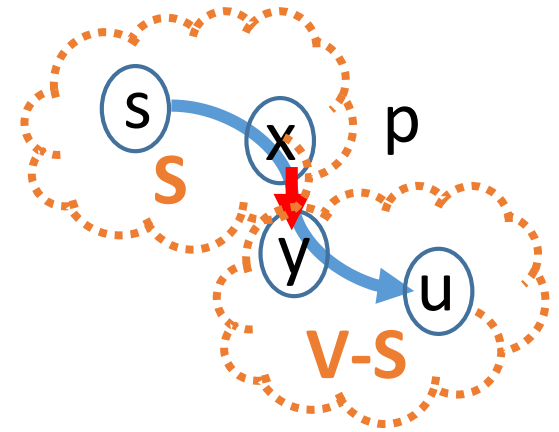
Dijkstra's Algorithm Correctness

Proof (cont.):

The path p connects a vertex in S (maybe s itself) to a vertex that is currently in $V-S$ (the vertex u).

Let y be the first vertex on p in $V-S$ and let x be its predecessor on the path p .

We claim that when y was inserted to S , we had:
 $d[y] = \delta(s, y)$.



SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

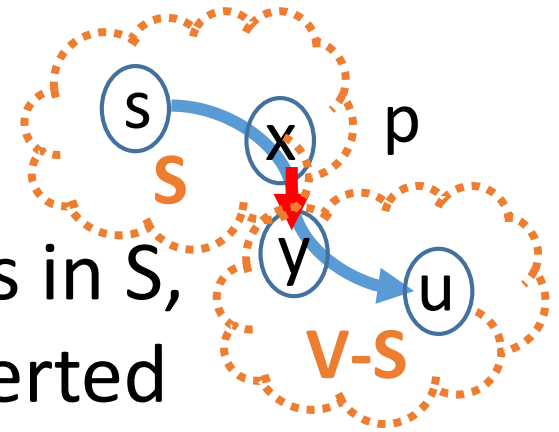
Proof (cont.):

The path p connects a vertex in S (maybe s itself) to a vertex that is currently in $V-S$ (the vertex u).

Let y be the first vertex on p in $V-S$ and let x be its predecessor on the path p .

We claim that when y was inserted to S , we had:

$d[y] = \delta(s, y)$. This is because its predecessor x was in S , so, $d[x] = \delta(s, x)$ (why?), but then, when x was inserted to S , a RELAX call was performed on all its outgoing edges, including (x, y) . So claim holds by property **3** of RELAX.



SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

Proof (cont.):

We can now reach contradiction.

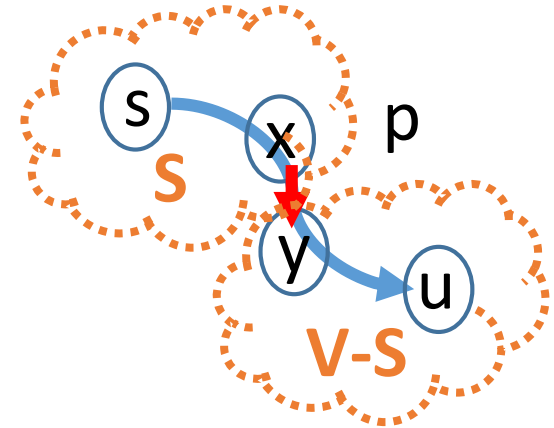
Since y appears before u in a shortest path from s to u , we have that:

$$\textcircled{1} \quad \delta(s, y) \leq \delta(s, u)$$

Therefore,

$$\textcircled{2} \quad d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u].$$

For $\textcircled{1}$ we use weights non-negativity!



SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

Proof (cont.):

We can now reach contradiction.

Since y appears before u in a shortest path from s to u , we have that:

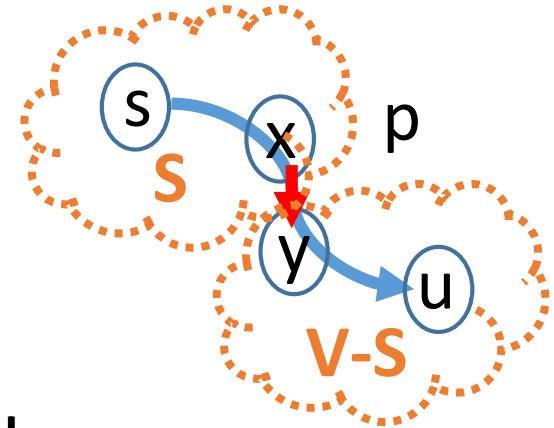
$$\textcircled{1} \quad \delta(s, y) \leq \delta(s, u)$$

Therefore,

$$d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u]. \quad \textcircled{2}$$

However, u and y were both in the Priority-Queue when u was extracted in line 5 as minimum, so it must be that: $d[u] \leq d[y].$ $\textcircled{3}$

For $\textcircled{1}$ we use weights non-negativity!



SSSP- Single Source Shortest Paths

Dijkstra's Algorithm Correctness

Proof (cont.):

We can now reach contradiction.

Since y appears before u in a shortest path from s to u , we have that:

$$\textcircled{1} \quad \delta(s, y) \leq \delta(s, u)$$

Therefore,

$$d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u]. \quad \textcircled{2}$$

However, u and y were both in the Priority-Queue when u was extracted in line 5 as minimum, so it must be that: $d[u] \leq d[y]$. $\textcircled{3}$

Therefore, $d[y] = \delta(s, y) = \delta(s, u) = d[u]$. Contradiction to the choice of u .

For $\textcircled{1}$ we use weights non-negativity!

